CCNA Exam Prep: Back to Networking Basics with Hank Preston and Patrick Gargano

A CCNA Diagnostic Playbook: Isolating Faults from Layer 1 to Layer 7

When network connectivity breaks, where do you begin? Troubleshooting is a cornerstone of the CCNA v2.0 exam, and having a systematic diagnostic methodology is what separates guessing from true engineering. In this live stream, Hank and Patrick build a practical diagnostic playbook from the ground up. We'll explore structured troubleshooting models using the OSI stack, highlighting core diagnostic tools, including `show` commands, syslog inspection, standard and extended ping, and traceroute. We'll also analyze pre-staged packet capture output to interpret real-world protocol behaviors (ARP, DHCP, and 802.1Q tags) without getting bogged down in capture setup. Join us to sharpen your troubleshooting instincts!

> Blueprint Topics:
>
> - 1.1: Diagnose interface and cable issues (collisions, errors, duplex, speed)
> - 1.6 Troubleshoot wired and wireless client connectivity (IP configuration, network reachability, and wireless security parameters on Windows, macOS, and Linux)
> - 2.4: Troubleshoot basic Layer 2/Layer 3 connectivity and device operations using show commands (including show logs), ping, extended ping, traceroute, and packet capture output
>
> Note: A PDF version of this lab guide is available on the CML Community.

Lab Requirements

This CML topology and lab require a CML server that supports public Internet access through the external connector named "Internet". It is configured in NAT mode to provide reachability to Internet resources through the CML host. This access is required to configure the "server" node and to complete some of the lab exercises.

If your CML server is "air-gapped" and doesn't have Internet access for topologies, this lab will not work as designed.

Setup and Scenario

This is a simplified network topology connecting a "client" and a "server" that is used as the basis for a series of "trouble tickets": scenarios where some network-based traffic is not working as expected.

The "server" is an Alpine Linux host, reachable at IP address 10.80.15.3. It acts as a DNS server for the domain "example.com" and provides recursive lookups to public DNS servers. It also represents a "web application server" hosting files that the "client" should be able to access.

The "client" is an Alpine Linux host and uses DHCP from "sw2" to configure its IP address. See `/etc/network/interfaces`.

Pre-validation

After starting the lab topology, verify the starting state by running these commands on "server".

> These commands verify that the startup configuration for "server" was completed successfully. This includes installing the DNS and HTTP services used in this lab.

```
server:~$ dig server.example.com

; <<>> DiG 9.20.29 <<>> server.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 23149
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 452d18fa4c4598ab010000006acfeb1709166434033ffc57 (good)
;; QUESTION SECTION:
;server.example.com.            IN      A

;; ANSWER SECTION:
server.example.com.     86400   IN      A       10.80.15.3

;; Query time: 1 msec
;; SERVER: 10.80.15.3#53(10.80.15.3) (UDP)
;; WHEN: Wed Oct 14 20:50:31 UTC 2026
;; MSG SIZE  rcvd: 91
```

```
server:~$ wget -O - http://server/test.txt
Connecting to server (127.0.0.1:80)
writing to stdout
Success!
-                    100% |********************************|     9  0:00:00 ETA
written to stdout
```

Trouble Tickets

The following troubleshooting tickets can be completed using the CML topology after it starts. Each ticket is self-contained; however, they should be completed in order because later tickets may "assume" that previous tickets have already been solved.

Troubleshooting walkthroughs for each ticket are provided, but initially collapsed/hidden so you can try to solve the ticket on your own.

Trouble Ticket #1: Client / Server Not Making a Connection

Title: A newly deployed "client" can't ping its "server"

Description: The team just deployed a new "client" host at a new site. The "client" is connected to "sw2" on port `e1/0`. It needs to be able to ping "server" (`10.80.15.3/29`) which is connected to "sw1". Both hosts are on the "APPLICATION" network.

Troubleshooting Method Suggestion: Bottom Up / Hop-by-Hop

Ticket Goal: Identify and fix the issue preventing "client" from pinging "server"

▼ **Troubleshooting Walkthrough**

We will tackle this ticket starting at the new "client" host, using a combination of hop-by-hop and bottom-up methodologies.
1. From the "client" and "sw2" consoles, check the network connection state (layer 1).

```
client:~$ ip link

1: lo:  mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0:  mtu 1500 qdisc pfifo_fast state UP qlen 1000
```

```
    link/ether 52:54:00:a4:78:3d brd ff:ff:ff:ff:ff:ff
```

```
sw2# show int e1/0 status

Port         Name               Status       Vlan       Duplex  Speed Type
Et1/0        Connection to clie connected    15           full   auto 10/100/10
```

Link shows "up" on client and "connected" on switch. Looks good. 1. Check whether the MAC address for "client" is learned on the switch (Layer 2).

```
sw2# show mac address-table interface ethernet 1/0
          Mac Address Table
------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----
15    5254.00a4.783d    DYNAMIC     Et1/0
Total Mac Addresses for this criterion: 1
```

> *Note: The MAC address shown on your lab instance will differ from this output. Check the MAC from your "client" to the "switch".* The MAC address from the "client" was learned by the switch on the expected interface.
1. Check whether "client" has an IP address (Layer 3).

```
client:~$ ip address show dev eth0
2: eth0:  mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 52:54:00:a4:78:3d brd ff:ff:ff:ff:ff:ff
    inet 10.80.15.5/29 scope global eth0
    valid_lft forever preferred_lft forever
    inet6 fe80::5054:ff:fea4:783d/64 scope link
    valid_lft forever preferred_lft forever
```

It does have an IP address. It is in the same subnet as "server" `10.80.15.3/29` <- *Server* `10.80.15.5/29` <- Client *So communication between client and server is L2 traffic 1. Try to ping "server" at its IP address*

```
client:~$ ping -c 4 10.80.15.3
PING 10.80.15.3 (10.80.15.3): 56 data bytes

--- 10.80.15.3 ping statistics ---
```

```
4 packets transmitted, 0 packets received, 100% packet loss
```

Ping not working. 1. Check ARP table on "client" for "server".

```
client:~$ ip neigh show
10.80.15.3 dev eth0  used 0/0/0 probes 6 FAILED
```

ARP failure 1. Start checking Layer 2 path hop-by-hop. 1. Check VLAN status on "sw2".

```
sw2# show vlan brief

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Et0/2, Et0/3, Et1/1, Et1/2
                                                Et1/3
9    USERS-FLOOR2                     active
10   USERS-FLOOR3                     active
11   USERS-FLOOR4                     active
15   APPLICATION                      active    Et1/0
19   GUESTS-FLOOR2                    active
20   GUESTS-FLOOR3                    active
22   GUESTS-FLOOR4                    active
999  MGMT                             active
```

VLAN 15 is "APPLICATION". *The "client" interface is in the VLAN. 1. The "server" is connected to "sw1". Let's check how "sw1" is connected to "sw2".*

```
sw2# show cdp neighbors
Capability Codes: R - Router, T - Trans Bridge, B - Source Route Bridge
                  S - Switch, H - Host, I - IGMP, r - Repeater, P - Phone,
                  D - Remote, C - CVTA, M - Two-port Mac Relay

Device ID        Local Intrfce     Holdtme    Capability  Platform  Port ID
sw1              Eth 0/0           162             R S I  Linux Uni Eth 0/0
sw1              Eth 0/1           163             R S I  Linux Uni Eth 0/1

Total cdp entries displayed : 2

sw2# show etherchannel summary
Flags:  D - down        P - bundled in port-channel
.
Number of channel-groups in use: 1
Number of aggregators:           1

Group  Port-channel  Protocol    Ports
```

```
------+------------+----------+-----------------------------------------------
1      Po1(SU)         LACP        Et0/0(P)        Et0/1(P)
```

Two interfaces connect the switches *The interfaces are combined into a port-channel 1. Check the switchport trunk status on "sw2".*

```
sw2# show interface po1 trunk

Port          Mode             Encapsulation  Status        Native vlan
Po1           on               802.1q         trunking      1

Port          Vlans allowed on trunk
Po1           9-11,15,19-21,999

Port          Vlans allowed and active in management domain
Po1           9-11,15,19-20,999

Port          Vlans in spanning tree forwarding state and not pruned
Po1           9-11,15,19-20,999
```

The "APPLICATION" vlan is active on the trunk on "sw2" 1. Move to the next "hop", "sw1" 1. Start back at the "bottom" and check the status of the interfaces on "sw1".

```
sw1# show etherchan sum
Flags:  D - down        P - bundled in port-channel
.
        A - formed by Auto LAG


Number of channel-groups in use: 1
Number of aggregators:           1

Group  Port-channel  Protocol    Ports
------+-------------+-----------+-----------------------------------------------
1      Po1(SU)         LACP        Et0/0(P)        Et0/1(P)
```

Looks good 1. Move to Layer 2 and check the VLAN status on "sw1".

```
sw1# show vlan b

VLAN Name                             Status    Ports
---- -------------------------------- --------- -------------------------------
1    default                          active    Et0/3, Et1/1, Et1/2, Et1/3
```

```
9    USERS-FLOOR2                     active
10   USERS-FLOOR3                     active
11   USERS-FLOOR4                     active
15   APPLICATION                      active    Et1/0
19   GUESTS-FLOOR2                    active
20   GUESTS-FLOOR3                    active
22   GUESTS-FLOOR4                    active
999  MGMT                             active


sw1# show interface po1 trunk

Port          Mode             Encapsulation  Status        Native vlan
Po1           on               802.1q         trunking      1

Port          Vlans allowed on trunk
Po1           9-11,19-21,999

Port          Vlans allowed and active in management domain
Po1           9-11,19-20,999

Port          Vlans in spanning tree forwarding state and not pruned
Po1           9-11,19-20,999
```

VLAN 15 - APPLICATION - exists on the switch *But it is **not** active on the trunk.* 1. Proposed fix: Add VLAN 15 to the trunk link to "sw2" on "sw1".

```
sw1# conf t

sw1(config)# int po1
sw1(config-if)# switchport trunk allowed vlan add 15
sw1(config-if)# end
sw1#
```

1. Test connectivity from "client" to "server".

```
client:~$ ping -c 4 10.80.15.3
PING 10.80.15.3 (10.80.15.3): 56 data bytes
64 bytes from 10.80.15.3: seq=0 ttl=42 time=3.439 ms
64 bytes from 10.80.15.3: seq=1 ttl=42 time=2.259 ms
64 bytes from 10.80.15.3: seq=2 ttl=42 time=2.188 ms
64 bytes from 10.80.15.3: seq=3 ttl=42 time=2.458 ms

--- 10.80.15.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
```

```
round-trip min/avg/max = 2.188/2.586/3.439 ms
```

Success!

Resolution: The trunk link from "sw1" to "sw2" was missing the required VLAN on the "sw1" end. VLAN 15 was added to the trunk, and the "client" was able to ping the "server".

Trouble Ticket #2: Secure Remote Management Not Working

Title: Unable to connect to "rtr1" with SSH for management

Description: An audit of the network's secure management protocols found that "rtr1" (`10.80.0.1`) is reachable via Telnet, but SSH is not working. SSH version 2 must be enabled and use local username and password authentication. Telnet must be disabled.

> - Test access from "server" for this ticket.
> - All credentials on "rtr1" are set to `cisco`

Troubleshooting Method Suggestion: Top-Down

Ticket Goal: Identify and resolve the problem with SSH management. Also disable Telnet access.

▼ **Troubleshooting Walkthrough**

We will use a "top-down" methodology for this ticket, testing access from the "server" Linux host.
1. Start by verifying the initial access state from "server".

```
server:~$ telnet 10.80.0.1
Connected to 10.80.0.1

Entering character mode
Escape character is '^]'.



User Access Verification

Username: cisco
Password:
```

```
rtr1#
```

Telnet is working

```
server:~$ ssh cisco@10.80.0.1
ssh: connect to host 10.80.0.1 port 22: Connection refused
```

SSH connections are refused. This looks like an application level issue. 1. Let's check the transport settings on the VTY lines of "rtr1".

```
rtr1# show run | sec line vty

line vty 0 4
activation-character 13
login local
transport input telnet ssh
```

Both telnet and ssh are enabled 1. Check the SSH status on "rtr1".

```
rtr1# show ip ssh

SSH Disabled - version 2.0
%Please create an RSA key of atleast 3072 bits or an EC key of atleast 384 bits
```

SSH is disabled, and a message tells us to create a key. 1. Create the key on "rtr1".

```
rtr1# crypto key generate ec keysize 384
% Please define a domain-name first.

rtr1# crypto key generate rsa general-keys modulus 4096
% Please define a domain-name first.
```

Both EC (elliptic curve) and RSA (Rivest-Shamir-Adleman) key-generation attempts fail because no domain name is configured. 1. Configure a domain name on "rtr1". The domain name doesn't matter for the ticket, it is just required to create a key.

```
rtr1# conf t
Enter configuration commands, one per line.  End with CNTL/Z.
```

```
rtr1(config)# ip domain name example.com
rtr1(config)# exit

rtr1# crypto key generate ec keysize 384
The name for the keys will be: rtr1.example.com

rtr1# show crypto key mypubkey all
% Key pair was generated at: 17:58:29 UTC Oct 13 2026
Key name: rtr1.example.com
Key type: EC KEYS      p384 curve
Storage Device: private-config
Usage: Signature Key
Key is not exportable. Redundancy enabled.
Key Data:
30763010 06072A86 48CE3D02 0106052B 81040022 03620004 ED4933D5 F4EE0D97
A70BA7D7 1A7D1121 979B196E 34823986 F4DD3A32 836DD8D6 E0D96350 1133BB05
1FB169D9 2FA4093C 4A532892 652F706E 32EC904E DA8F2372 53974C19 FCDE32B1
17ED282B 46AA24F5 A0F68C5C 8D1FF4C4 5C4CCFEC 99F90514
```

1. Let's check the log. Key generation should generate a message.

```
rtr1# show log last 5
Syslog logging: enabled (0 messages dropped, 2 messages rate-limited, 0 flushes
.

Showing last 5 lines

Log Buffer (4096 bytes):

Oct 13 17:07:52.586: %SYS-5-CONFIG_I: Configured from console by console
Oct 13 17:18:15.378: %SYS-6-TTY_EXPIRE_TIMER: (exec timer expired, tty 0 (0.0.0
Oct 13 17:58:26.098: %SYS-5-CONFIG_I: Configured from console by console
Oct 13 17:58:29.709: %CRYPTO_ENGINE-5-KEY_ADDITION: A key named rtr1.example.co
Oct 13 17:58:29.718: %SSH-5-ENABLED: SSH 2.0 has been enabled
```

The key was created. SSH now shows as enabled. 1. Verify SSH status.

```
rtr1# show ip ssh
SSH Enabled - version 2.0
Authentication methods:publickey,keyboard-interactive,password
.
```

1. Check to see if SSH access works from "server".

```
server:~$ ssh cisco@10.80.0.1

(cisco@10.80.0.1) Password:


rtr1#
```

SSH is now working. We'll leave the whole "post-quantum key exchange" issue for another ticket :-) 1. Disable telnet on "rtr1"

```
rtr1# conf t
Enter configuration commands, one per line.  End with CNTL/Z.

rtr1(config)# line vty 0 4
rtr1(config-line)# transport input ssh
rtr1(config-line)# end
rtr1#
```

1. Test telnet from "server" to verify it doesn't work.

```
server:~$ telnet 10.80.0.1
telnet: can't connect to remote host (10.80.0.1): Connection refused
```

Resolution: SSH wasn't working on "rtr1" because no key was configured. A domain name had to be added to generate a key. SSH access from "server" was tested, then Telnet was disabled and verified.

Trouble Ticket #3: Client Configuration Download Failing

Title: Client unable to download config or data from "server"

Description: While setting up the application on "client", the systems team encountered an issue downloading the required files `http://server/config.file` and `http://server/data.file`. The test file `http://server/test.txt` does work.

> - The command `wget -T 10` can be used to download files. Ex: `wget -T 10 http://server/test.txt`

    - Note: The `-T 10` argument sets the timeout for `wget` to 10 seconds. The default is indefinite
- Determine why the file downloads are failing on "client"

Troubleshooting Method Suggestion: Top-Down

Ticket Goal: Diagnose the problem. Determine the most likely cause and suggest ways to resolve it. Actually fixing the problem is not necessary.

<details>
<summary>▼ Diagnostic Walkthrough</summary>

Trouble Ticket #1 verified basic connectivity between "client" and "server" using a bottom-up approach. This time, we'll start at the top and work our way down to find the issue.
1. First, replicate the problem on "client" by attempting to download all three files: the test file, config file, and data file.

```
client:~$ wget -T 10 http://server/test.txt
Connecting to server (10.80.15.3:80)
saving to 'test.txt'
test.txt             100% |**|     9  0:00:00 ETA
'test.txt' saved

client:~$ wget -T 10 http://server/config.file
Connecting to server (10.80.15.3:80)
wget: download timed out

client:~$ wget -T 10 http://server/data.file
Connecting to server (10.80.15.3:80)
wget: download timed out

client:~$ ls -lh
total 4K
-rw-r--r--    1 cisco    cisco          9 Oct 14 15:45 test.txt
```

Replicated the problem "client" can access and download the "test.txt" file successfully, but the other files fail. 1. Test that the downloads work from "server".

```
server:~$ wget -T 10 http://server/test.txt
Connecting to server (127.0.0.1:80)
saving to 'test.txt'
test.txt             100% |**|     9  0:00:00 ETA
'test.txt' saved
```

```
server:~$ wget -T 10 http://server/config.file
Connecting to server (127.0.0.1:80)
saving to 'config.file'
config.file          100% |**| 1024k  0:00:00 ETA
'config.file' saved


server:~$ wget -T 10 http://server/data.file
Connecting to server (127.0.0.1:80)
saving to 'data.file'
data.file            100% |**|  100M  0:00:00 ETA
'data.file' saved

server:~$ ls -lh
total 101M
-rw-r--r--    1 cisco    cisco       1.0M Oct 14 15:50 config.file
-rw-r--r--    1 cisco    cisco     100.0M Oct 14 15:50 data.file
-rw-r--r--    1 cisco    cisco          9 Oct 14 15:50 test.txt
```

The downloads are working fine on "server". 1. Top-Down analysis so far Layers 7-5 [Application, Presentation, Session] seem to be working from "client" to "server" because the test file downloads successfully. *Layer 4 [Transport] - Must be working in order for test file download to work* Layer 3 [Network] - No routing is involved between "server" and "client", but ping and reachability are working. *Layer 2 [Data Link] - With HTTP and ping working, layer 2 seems healthy* Layer 1 [Physical] - Seems to be working, but are there errors?
1. Let's check interface details on the switches connected to "server" and "client".

```
sw1# show inter e1/0
Ethernet1/0 is up, line protocol is up (connected)
Hardware is Ethernet, address is aabb.cc00.6301 (bia aabb.cc00.6301)
Description: Connection to server
MTU 1500 bytes, BW 10000 Kbit/sec, DLY 1000 usec,
    reliability 255/255, txload 1/255, rxload 1/255
Encapsulation ARPA, loopback not set
Keepalive set (10 sec)
.
Queueing strategy: fifo
Output queue: 0/40 (size/max)
5 minute input rate 0 bits/sec, 0 packets/sec
5 minute output rate 0 bits/sec, 0 packets/sec
    14268 packets input, 1365896 bytes, 0 no buffer
    Received 0 broadcasts (0 multicasts)
    0 runts, 203 giants, 0 throttles
    208 input errors, 0 CRC, 0 frame, 0 overrun, 5 ignored
    0 input packets with dribble condition detected
```

```
    78467 packets output, 47205719 bytes, 0 underruns
    Output 47293 broadcasts (47245 multicasts)
    2 output errors, 2 collisions, 1 interface resets
    0 unknown protocol drops
    0 babbles, 0 late collision, 0 deferred
    0 lost carrier, 0 no carrier
    0 output buffer failures, 0 output buffers swapped out
```

```
sw2# show int e1/0
Ethernet1/0 is up, line protocol is up (connected)
Hardware is Ethernet, address is aabb.cc00.6001 (bia aabb.cc00.6001)
Description: Connection to client
MTU 1500 bytes, BW 10000 Kbit/sec, DLY 1000 usec,
    reliability 255/255, txload 1/255, rxload 1/255
Encapsulation ARPA, loopback not set
Keepalive set (10 sec)
.
Queueing strategy: fifo
Output queue: 0/40 (size/max)
5 minute input rate 0 bits/sec, 0 packets/sec
5 minute output rate 0 bits/sec, 0 packets/sec
    205 packets input, 20601 bytes, 0 no buffer
    Received 0 broadcasts (0 multicasts)
    0 runts, 10 giants, 0 throttles
    10 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored
    0 input packets with dribble condition detected
    6791 packets output, 838563 bytes, 0 underruns
    Output 6634 broadcasts (5373 multicasts)
    0 output errors, 0 collisions, 0 interface resets
    0 unknown protocol drops
    0 babbles, 0 late collision, 0 deferred
    0 lost carrier, 0 no carrier
    0 output buffer failures, 0 output buffers swapped out
```

Both switches show input errors and giants on the interfaces connected to the hosts. "Giants" are packets that exceed an interface's MTU.

1. Check the MTU configured on the "server" and "client".

```
server:~$ ip link show dev eth0
2: eth0:  mtu 9000 qdisc pfifo_fast state UP qlen 1000
    link/ether 52:54:00:e4:a3:89 brd ff:ff:ff:ff:ff:ff
```

```
client:~$ ip link show dev eth0
2: eth0:  mtu 9000 qdisc pfifo_fast state UP qlen 1000
```

```
        link/ether 52:54:00:a9:40:25 brd ff:ff:ff:ff:ff:ff
```

Both hosts are configured for jumbo frames with a 9000-byte MTU. The switch interfaces use a standard MTU of 1500 bytes. **Diagnosis:** An MTU mismatch between the hosts and the network is causing issues with "bulky" data transmissions. The "test file" is only 9 bytes and doesn't exceed the standard MTU on the switches. However, the config and data files are larger, and the hosts are sending network traffic that is too large for the switches. To resolve the issue, the hosts' MTUs must be no larger than the MTU supported by the switches in the path. We can test the diagnosis with ping by setting the size and do-not-fragment bits.
1. From "client", send a `1472-byte` ping to test the standard MTU by sending a 1500-byte packet.

```
client:~$ ping -c 4 -s 1472 server
PING server (10.80.15.3): 1472 data bytes
1480 bytes from 10.80.15.3: seq=0 ttl=42 time=2.268 ms
1480 bytes from 10.80.15.3: seq=1 ttl=42 time=2.583 ms
1480 bytes from 10.80.15.3: seq=2 ttl=42 time=2.736 ms
1480 bytes from 10.80.15.3: seq=3 ttl=42 time=2.601 ms

--- server ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 2.268/2.547/2.736 ms
```

* Note: The ICMPv4 header is 8 bytes, and the IPv4 header is 20 bytes.

```
        1472 (data) + 8 (ICMP) + 20 (IPv4) = 1500 bytes
```

1. Now, from "client", send a "jumbo" ping with a total size larger than 1500 bytes.

```
client:~$ ping -c 4 -s 2000 server
PING server (10.80.15.3): 2000 data bytes

--- server ping statistics ---
4 packets transmitted, 0 packets received, 100% packet loss
```

Diagnosis verified!

▼ **Fixing the Issue!**

This ticket was about diagnosing the problem, but you may want to fix it too! Me too :-) The problem is that the hosts are using a larger MTU than the network supports. We either need to increase the network's MTU or decrease the MTU on the hosts. > Note: Having a network with a larger MTU than its hosts is typically not a problem. This configuration is common in a data center, where the switches would be configured for jumbo frames and server engineers could choose to use jumbo frames on a server-by-server basis.
The IOL switches used in this lab don't support changing the MTU on the network, but we can change the MTU on the Linux hosts using the `ip link` command.
Run this command on "client". No reboot is necessary.

```
sudo ip link set dev eth0 mtu 1500
```

Now try to download the config or data file.

```
client:~$ wget http://server/config.file
Connecting to server (10.80.15.3:80)
saving to 'config.file'
config.file          100% |**| 1024k  0:00:00 ETA
'config.file' saved
```

Success! You may wonder why we didn't need to change the MTU on the server. That's a great question. During the four-way handshake for TCP, hosts negotiate a maximum segment size. If two hosts support different MTUs, they will agree to use the largest MTU supported by both.

Trouble Ticket #4: Internet is down!

> Note: We got a little excited about the topic, and this lab guide got a little too big to embed within CML. You can find Ticket 4 in the PDF lab guide available with the resources on the CML Community.

Title: "Client" unable to access Internet websites

Description: The "client" can now access "server" resources; however, it can't reach any public Internet services, even though it should be able to.

> - Use "ping" to validate access to resources

> - You can validate Internet access using hosts `u.cisco.com` or `cisco.com`
> - If your lab environment blocks ICMP traffic to the Internet, you can use `csnake.example.com` and `quantumcloud.example.com` as alternatives. (These addresses point to the "server" host)

Troubleshooting Method Suggestion: Divide and Conquer + Packet Capture

Ticket Goal: Find and fix the problem preventing Internet access for "client".

<details>
<summary>▼ Troubleshooting Walkthrough</summary>

We'll use the divide and conquer approach for this ticket!
1. Start with some basic ping checks from "client". We'll test access to `u.cisco.com`.

```
client:~$ ping u.cisco.com
ping: bad address 'u.cisco.com'
```

Replicated the issue. Good start 1. Let's try some well-known public servers, such as 1.1.1.1 and 8.8.8.8 (IP addresses for Internet DNS servers).

```
client:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=42 time=5.065 ms
64 bytes from 8.8.8.8: seq=1 ttl=42 time=3.947 ms
64 bytes from 8.8.8.8: seq=2 ttl=42 time=3.180 ms
64 bytes from 8.8.8.8: seq=3 ttl=42 time=3.403 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 3.180/3.898/5.065 ms


client:~$ ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1): 56 data bytes
64 bytes from 1.1.1.1: seq=0 ttl=42 time=9.216 ms
64 bytes from 1.1.1.1: seq=1 ttl=42 time=7.799 ms
64 bytes from 1.1.1.1: seq=2 ttl=42 time=7.811 ms
64 bytes from 1.1.1.1: seq=3 ttl=42 time=7.335 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
```
</details>

```
round-trip min/avg/max = 7.335/8.040/9.216 ms
```

IP reachability to the Internet is working 1. Divide and Conquer analysis - Layer 3 looks good, but DNS isn't working. We'll move up the stack. 1. Check the DNS configuration on the Linux host. Check the contents of the file `/etc/resolv.conf`.

```
client:~$ cat /etc/resolv.conf
cat: can't open '/etc/resolv.conf': No such file or directory
```

No file or DNS configuration 1. Check the network interface configuration script at `/etc/network/interfaces`

```
client:~$ cat /etc/network/interfaces

auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
```

"client" is using DHCP for IP configuration Analysis so far: *This looks like a DNS problem: there isn't a DNS server configured on "client".* DHCP is used to configure an IP address on "client". Options: *Statically configure the DNS server on "client" as 10.80.15.3.* Investigate the DHCP service to see what is being offered with the IP address Let's use the suggested packet capture to check out DHCP. 1. Start a packet capture in CML on the interface between "client" and "sw2". *Configure the capture to include only DHCP packets using the filter `udp port 67 or port 68`. 1. Restart the networking service on "client" to restart DHCP.*

```
client:~$ sudo service networking restart
 WARNING: you are stopping a boot service
 Stopping networking ...
   lo ...
[ ok ]
   eth0 ...
[ ok ]
 Starting networking ...
   lo ...
[ ok ]
   eth0 ...
udhcpc: started, v1.37.0
```

```
udhcpc: broadcasting discover
udhcpc: broadcasting select for 10.80.15.6, server 10.80.15.2
udhcpc: lease of 10.80.15.6 obtained from 10.80.15.2, lease time 86400
[ ok ]
```

1. Stop the packet capture in CML 1. Find and inspect the "DHCP Offer" packet. Look at the included options. *You should see* `Option: (15) Domain Name` However, `Option: (6) Domain Name Server` is missing. *Make note of the DHCP server address:* `10.80.15.2` *("sw2") 1. Look at the DHCP pool configuration on "sw2".*

```
sw2# show run | sec dhcp
ip dhcp excluded-address 10.80.15.1 10.80.15.4
ip dhcp pool APPLICATION
network 10.80.15.0 255.255.255.248
default-router 10.80.15.1
domain-name example.com
```

As expected, no DNS server is configured. 1. Let's add the DNS server configuration

```
sw2# conf t
Enter configuration commands, one per line.  End with CNTL/Z.

sw2(config)# ip dhcp pool APPLICATION
sw2(dhcp-config)# dns-server 10.80.15.3
```

1. Clear and restart the packet capture. (You might need to reconfigure the BPF filter.) 1. Restart the networking service on "client". 1. Stop the packet capture. 1. Find the new DHCP Offer packet and check the options. *You should now see* `Option: (6) Domain Name Server`*. 1. Test Internet access again on "client".*

```
client:~$ ping -c 4 u.cisco.com
PING u.cisco.com (108.156.224.43): 56 data bytes
64 bytes from 108.156.224.43: seq=0 ttl=42 time=8.663 ms
64 bytes from 108.156.224.43: seq=1 ttl=42 time=7.892 ms
64 bytes from 108.156.224.43: seq=2 ttl=42 time=7.688 ms
64 bytes from 108.156.224.43: seq=3 ttl=42 time=7.058 ms

--- u.cisco.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 7.058/7.825/8.663 ms
```

Success! **Resolution:** The "client" lacked a DNS server configuration because DHCP on "sw2" was not configured to provide one. The DHCP pool was updated to provide the DNS server, and the DHCP lease was renewed. Internet access was then tested with `ping u.cisco.com`.